T2

Learning from Data

Salvador Ruiz Correa

October 19th , 2026

Agenda:
1 Learning from data.
2 Supervised learning.

Machine learning is a subfield of artificial intelligence (AI) that focuses on developing algorithms and statistical models that enable computer systems to learn from and make predictions or decisions based on data. In essence, machine learning is the science of allowing computers to automatically learn and improve their performance on a specific task over time without being explicitly programmed for that task.

Learning from Data

Learning from data refers to deriving insights, models, or solutions based on data, especially in cases where a precise analytical or theoretical solution is either unavailable or too complex to construct. Instead of relying on pre-existing equations or models, we leverage collected data to create an empirical solution that captures patterns and behaviors inherent in the data itself.

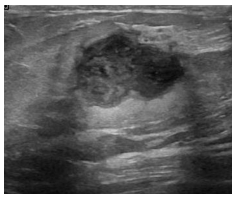


Figure 1: Is there evidence of breast cancer in this ultrasound image?

This approach is remarkably versatile, covering many applications across scientific disciplines, engineering, and beyond. In science, learning from data enables researchers to identify trends and make predictions in complex systems, from climate modeling to genetic research, where data can reveal insights that are difficult to deduce theoretically. Engineering underpins robotics, autonomous systems,

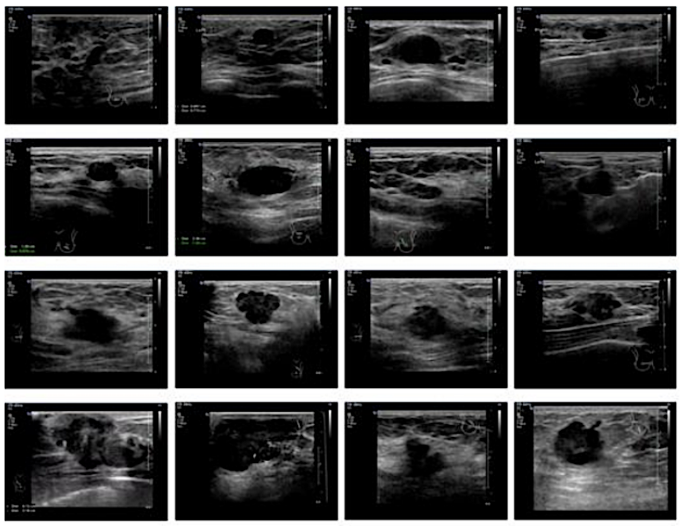


Figure 2: Examples of abnormal breasts ultrasounds.

and control engineering advancements, allowing machines to improve performance through data-driven learning algorithms. This method also has broad applications in medicine, finance, and social sciences, where large datasets can reveal associations and predictive patterns that aid in decision-making, diagnosis, and resource allocation.

Overall, learning from data is a foundational approach to solving real-world problems. Its adaptability to the availability and richness of data reassures us of its applicability, making it one of the most widely used techniques in modern problem-solving across diverse fields.

The Supervised Learning Problem

If you show a picture to a three-year-old and ask if it includes a tree, you'll likely get the correct answer. But if you ask a thirty-year-old to define a tree, the response might be vague or inconclusive. We don't learn what a tree is by studying its formal definition; we learn by seeing trees. In other words, we learn from data. Similarly, a radiology student requires extensive training hours to identify abnormalities in imaging studies. Viewing images is essential to a radiologist's education (Figures 1 and 3).

The core idea of learning from data involves using a set of observations to reveal an underlying process. This broad concept does not easily fit within a single framework. Consequently, various learning paradigms have emerged to address different scenarios and assumptions. In this section, we introduce several of these paradigms. The learning approach we have covered is supervised learning, the most studied and commonly applied type. However, it is not the only approach. Some variations of supervised learning are straightforward enough to fit within the same framework. In contrast, others are more complex and introduce new concepts and techniques that become distinct areas of study. The most significant variations stem from the nature of the dataset.

The focus of supervised learning is on the problem of prediction. Given a sample of training examples

$$(x_1, y_1), \ldots, (x_n, y_n)$$

from $\mathbb{R}^d \times \mathbb{R}$, we learn a predictor $h_n : \mathbb{R}^d \to \mathbb{R}$ that is used to predict the label y of a new point x, unseen in training.

The predictor h_n is commonly chosen from some function class $\mathcal{H}$, such as neural networks with a certain architecture, using empirical risk minimization (ERM) and its variants. In ERM, the predictor is taken to be a function $h \in \mathcal{H}$ that minimizes the empirical (or

training) risk $\mathcal{R}(h) = \frac{1}{n}\sum_{i=1}^{n} l(h(x_i), y_i)$, where l is a loss function, such as the squared loss $l(y', y) = (y' - y)^2$ for regression or 0-1 loss $l(y', y) = \mathbf{1}_{\{y' \neq y\}}$ for classification.

The goal of machine learning is to find h_n that performs well on new data, unseen in training. To study performance on new data (known as generalization), we typically assume the training examples are sampled randomly from a probability distribution P over $\mathbb{R}^d \times \mathbb{R}$ and evaluate h_n on a new test example (x, y) drawn independently from P. The challenge stems from the mismatch between the goals of minimizing the empirical risk (the explicit goal of ERM algorithms, optimization) and minimizing the true (or test) risk $\mathbb{E}_{(x,y)\sim P}[l(h(x), y)]$ (the goal of machine learning).

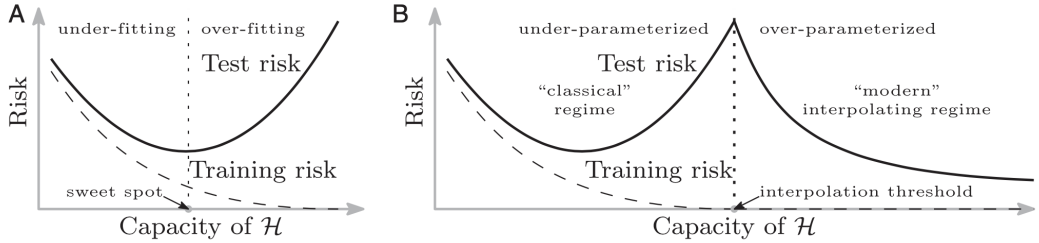


Figure 3: Curves for training risk (dashed line) and test risk (solid line). (A) The classical U-shaped risk curve arising from the bias–variance trade-off. (B) The double-descent risk curve, which incorporates the U-shaped risk curve (i.e., the "classical" regime) together with the observed behavior from using high- capacity function classes (i.e., the "modern" interpolating regime), separated by the interpolation threshold. The predictors to the right of the interpolation threshold have zero training risk. Source: https://doi.org/10.1073/pnas.1903070116 (2019).

Conventional wisdom in machine learning suggests controlling the capacity of the function class $\mathcal{H}$ based on the bias–variance trade-off by balancing underfitting and overfitting:

1. If $\mathcal{H}$ is too small, all predictors in $\mathcal{H}$ may underfit the training data (i.e., have large empirical risk) and hence predict poorly on new data.
2. If $\mathcal{H}$ is too large, the empirical risk minimizer may overfit spurious patterns in the training data, resulting in poor accuracy on new examples (small empirical risk but large true risk).

The classical thinking is concerned with finding the "sweet spot" between underfitting and overfitting. The control of the function class capacity may be explicit, via the choice of $\mathcal{H}$ (e.g., picking the neural network architecture), or it may be implicit, using regularization (e.g., early stopping). When a suitable balance is achieved, the performance of h_n on the training data is said to generalize to the population P. This is summarized in the classical U-shaped risk curve shown in Fig. 1A that has been widely used to guide model selection and is even thought to describe aspects of human decision making (3). The textbook corollary of this curve is that "a model with zero training error is overfit to the training data and will typically generalize poorly", a view still widely accepted.

However, practitioners routinely use modern machine-learning

methods, such as large neural networks and other nonlinear predictors that have very low or zero training risk. Despite the high function class capacity and near-perfect fit to training data, these predictors often give very accurate predictions on new data. Indeed, this behavior has guided a best practice in deep learning for choosing neural network architectures, specifically that the network should be large enough to permit effortless zero-loss training (called interpolation) of the training data (4). Moreover, in direct challenge to the bias–variance trade-off philosophy, recent empirical evidence indicates that neural networks and kernel machines trained to interpolate the training data obtain near-optimal test results even when the training data are corrupted with high levels of noise.

Remark. You can find discussion of these ideas in the paper by M. Belkin,D. Hsu,S. Ma, & S. Mandal: https://doi.org/10.1073/pnas.1903070116 (2019). The previous text was taken from this article.

Supervised Learning Roadmap

The key supervised learning concepts are the following.

Symmary 1: Key Supervised Learning Concepts

1. The learning problem.
2. Generalization Theory.
3. Generalization Bound Interpretation.
4. Approximation-Generalization Tradeoff.
   (a) Bias-Variance.
   (b) Learning Curve.
   (c) Learning Regimes.
5. Overfitting
   (a) Regularization.
   (b) Validation set (validation errorE_{val})
6. Sampling bias.

Supervised Learning Overview

Supervised learning is a function estimation procedure in which the goal is to learn a function that best maps inputs to desired outputs based on a labeled dataset (Figure 4). In this setup, we are given a set of data points, each consisting of an input (often represented as a vector) and an associated output label. The aim is to estimate a function f that, when given a new input, can accurately predict its corresponding output based on patterns it has learned from the

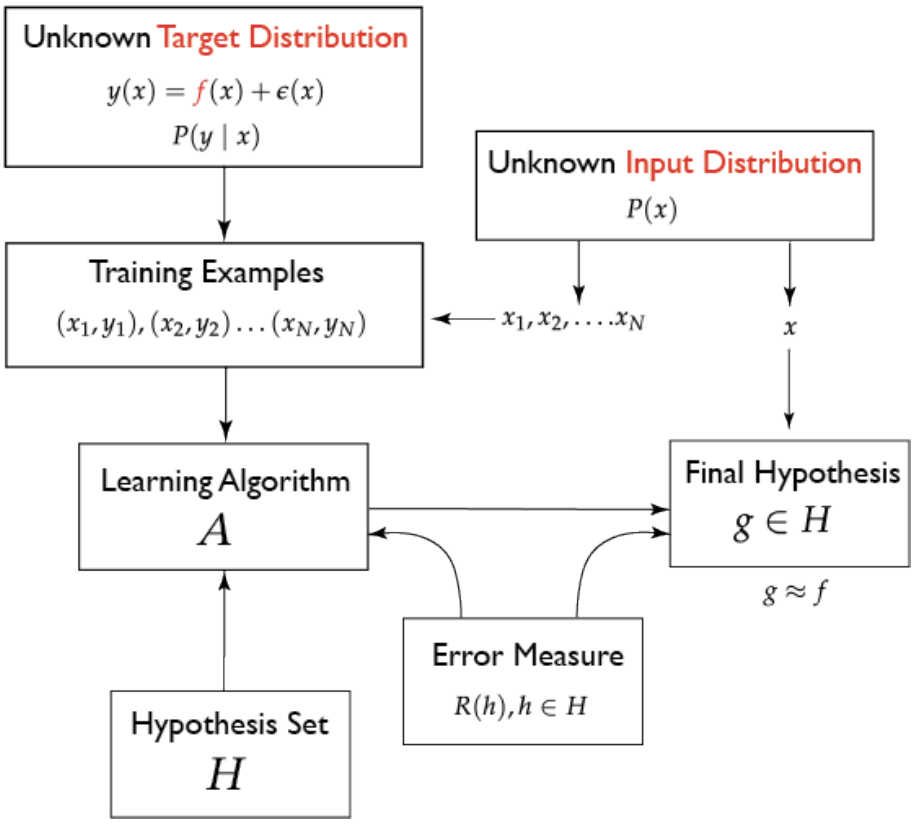


Figure 4: General supervised learning problem. This problem considers noisy targets; that is, the target y is a function $y(x)$ of a deterministic component ($f(x)$) and a random component ($\epsilon(x)$).

labeled examples.

Theoretically, supervised learning seeks to approximate the underlying relationship between inputs and outputs by finding an approximation function. This approximation is achieved by minimizing the difference between the predicted outputs and the actual outputs in the training data, typically using a loss function such as *mean squared error* for regression tasks or *cross-entropy for classification tasks*. The learning algorithm iteratively adjusts the function's parameters to reduce this loss, resulting in a model that generalizes well to new, unseen data.

This function estimation approach is foundational in supervised learning. It frames the problem as finding a generalizable mapping from inputs to outputs rather than simply memorizing the training examples. In doing so, it allows the model to make accurate predictions for new inputs by estimating the likely output values based on learned patterns, effectively serving as an empirical approximation to the actual function governing the data.

Components of Learning

To abstract the common core of the learning problem, we will pick an application and use it as a metaphor for the different components of the problem.

Imagine a bank that receives thousands of credit card applications daily and aims to automate the evaluation process. While there isn't a magical formula for determining credit approval, the bank possesses a wealth of data. This scenario is perfect for utilizing *data-driven learning* (i.e. learning for data). To devise a reliable formula for credit approval, the bank analyzes historical data records of previous customers.

Each customer record includes personal information pertinent to credit, such as age, income, years of residence, outstanding loans, and more. Additionally, the records indicate whether granting credit to a particular customer was profitable for the bank. This data helps in creating an effective formula for credit approval, which can then be applied to future applicants.

Names and symbols to the primary components of the learning problem are as follows.

- We have the input x, representing the customer information used to make the credit decision.
- The unknown target function is $f : \mathcal{X} \to \mathcal{Y}$, where $\mathcal{X}$ is the *input or feature space* (the set of all possible inputs) and $\mathcal{Y}$ is the *output space* (the set of all possible outputs, in this case, the yes/no decision).
- There is a data set $\mathcal{D} = \{(x_1, y_1), \ldots, (x_N, y_N)\}$, where $y_n = f(x_n)$ for $n = 1, \ldots, N$ (inputs corresponding to previous customers and the correct decision for them in hindsight). $\mathcal{D}$ is called the *training data set*. The examples are often referred to as *data points*.
- Finally, the learning algorithm uses the data set $\mathcal{D}$ to pick the formula $g : \mathcal{X} \to \mathcal{Y}$ that approximates f. The function g is chosen from a set of candidate formulas, or *hypothesis set* $\mathcal{H}$.

When a new customer applies for credit, the bank will base its decision on g (the hypothesis that the learning algorithm produced), not f (the ideal target function remains unknown).

The decision will be good only to the extent the g faithfully replicates f. To achieve that, the algorithm chooses g that best matches f on the *training examples* of previous customers, hoping it will continue to match f on new customers. Whether or not this hope is justified is a matter of careful analysis.

A principled approach to tackle the learning problem, i.e., finding a faithful approximation of f that *generalizes* on new data, requires that we assume that data points are sampled from a joint *probability distribution function* $P(x)$ (Figure 4).

[o] Source:Yaser S. A-M, Malik M-I, Hsuan-Tien L.., Learning from Data, A short Course, AMLbook.com.

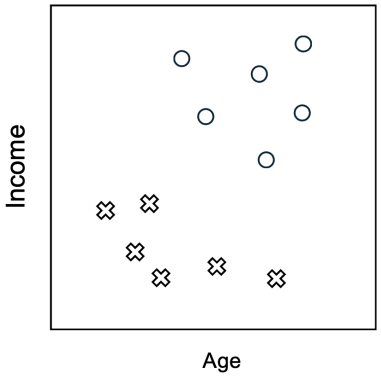


Figure 5: Data set $\mathcal{D} = \{(x_1, y_1), \ldots, (x_N, y_N)\}$, where $y_n = f(x_n)$ for $n = 1, \ldots, N$.

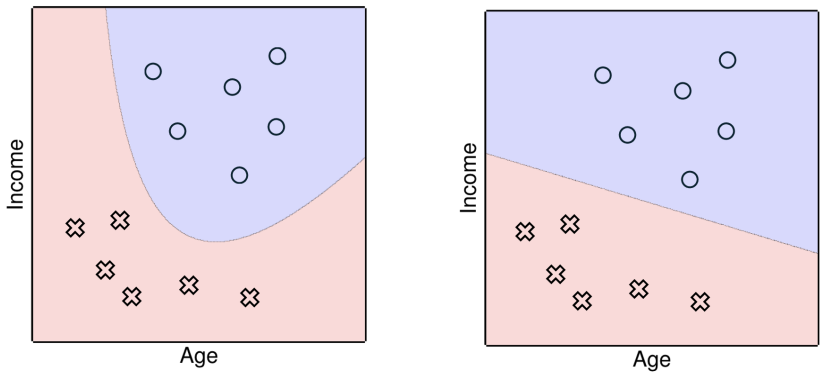


Figure 6: Our goal is to use $\mathcal{D}$ to find a function $g \in \mathcal{H}$ that approximates f and generalizes over new data. The graph on the left shows the f function, and the graph on the right an hypothesis $h \in \mathcal{H}$.

Binary Classification Problem

We are given a training set $\mathcal{D} = \{(x_1, y_1), \ldots, (x_N, y_N)\}$, where $y \in \{1, -1\}$, $y_n = f(x_n)$, for $n = 1, \ldots, N$. We aim to use $\mathcal{D}$ to find a function $g \in \mathcal{H}$ that approximates f and generalizes over new data.

Given $h \in \mathcal{H}$, we define the error rate within the sample or *in-sample error* (i.e. the fraction of $\mathcal{D}$ where f and h disagree) is defined as

$$E_{\text{in}}(h) = \frac{1}{N} \sum_{n=1}^{N} \mathrm{I}\left[h(x_n) \neq f(x_n)\right],$$

where $\mathrm{I}[\text{statement}] = 1$ if the statement is true, and equal to 0 if the statement is false. We have made the dependency of $E_{\text{in}}(h)$ on the particular h we are considering. In the same way, we define the out-of-sample error.

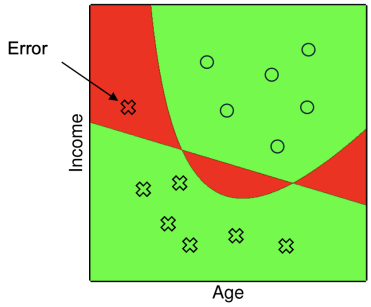


Figure 7: Calculation of E_{in}.

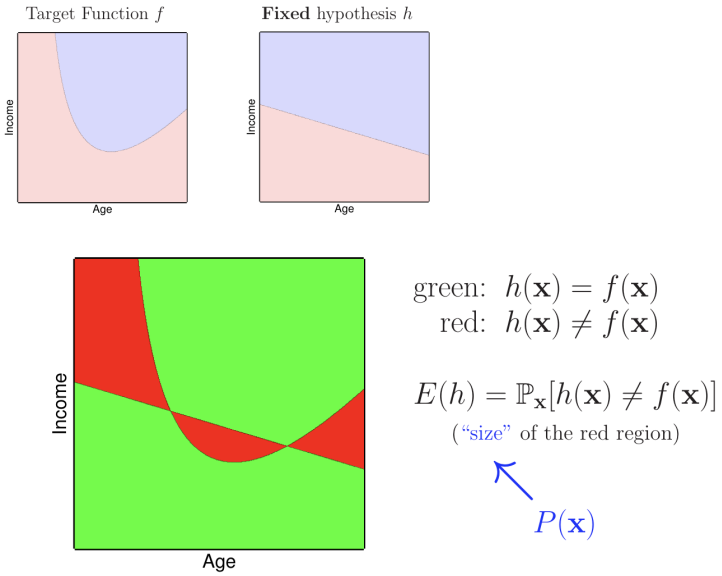


Figure 8: Calculation of E_{out}.

$$E_{\text{out}}(h) = P(h(x_n) \neq f(x_n)),$$

where the probability is based on the distribution P over $\mathcal{X}$, which is used to sample the data points x.

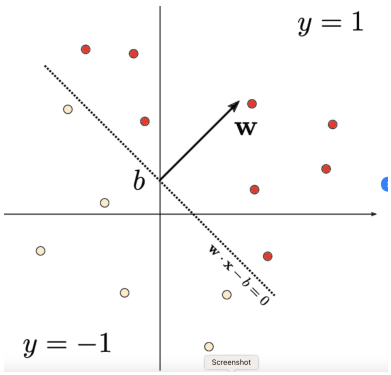


Figure 9: Separating hyperplane.

Example: Linear Binary Classifier

The linear model for classifying data into two classes uses a hypothesis set of linear classifiers. The hypothesis space is defined as

$$\mathcal{H} = \{h(x) : h(\mathbf{x}) = \text{sign}(\mathbf{w}^T\mathbf{x}), \mathbf{w} \in R^{d+1}\},$$

where d is the dimensionality of the input space. The added coordinate $x_0 = 1$ corresponds to the bias weight w_0. More specifically: $\mathbf{x} = (1, x_1, x_2, \ldots x_d)$ and $\mathbf{w} = (w_0, w_1, x_2, \ldots w_d)$. Recall that

$$h(\mathbf{x}) = \begin{cases} 1 & \text{if } \sum_{i=0}^{d+1} w_i x_i > 0, \\ 0. & \text{otherwise.} \end{cases}$$

Feasibiity of Learning

Given a tolerance level δ, for example $\delta = 0.05$, and assert with probability at least $1 - \delta$ that

$$E_{\text{out}}(g) \leq E_{\text{in}}(g) + \sqrt{\frac{1}{2N} \ln \frac{2M}{\delta}}. \tag{2.1}$$

M is the cardinality of the hypothesis space.

Ideally, the learning algorithm should lead to an approximation of f (i.e. g) that generalizes over new data points. This means that $E_{\text{out}}(g) \approx 0$. However, the stated probabilistic bound only provides the condition under which $E_{\text{in}}(g) \approx E_{\text{out}}(g)$. We must ascertain whether $E_{\text{in}}(g) \approx 0$.

In other words, we have traded the condition $E_{\text{out}}(g) \approx 0$ by the conditions $E_{\text{in}}(g) \approx E_{\text{out}}(g)$ and $E_{\text{in}}(g) \approx 0$. In some applications, we can relax the condition $E_{\text{in}}(g) \approx 0$ by allowing a small error, where the definition of "small" is application-dependent.

The feasibility of learning is thus split into two questions:

Definition 1: Feasibility of Learning Questions

1. Can we make sure that $E_{\text{out}}(g)$ is close enough to $E_{\text{in}}(g)$?
2. Can we make sure that $E_{\text{in}}(g)$ small enough?

Breaking down the feasibility of learning into these two questions provides further insights into the role of the learning problem's different components. One such component concerns the *complexity* of these components, which relates to the generalization theory.

The most crucial theorem in generalization theory is the so-called VC generalization bound, which replaces the bound used in our previous calculation. The new bound no longer depends on M, but a new function called the growth function of $\mathcal{H}$. Alternatively, we can use the so-called VC dimension. We remark that this theorem *establishes the feasibility of learning with infinite hypothesis tests*.

Box 1: VC generalization bound

- For any $\delta > 0$,

$$E_{\text{out}}(g) \leq E_{\text{in}}(g) + \sqrt{\frac{8}{N} \log \frac{4m_{\mathcal{H}}(2N)}{\delta}}, \tag{1}$$

  with probability $1 - \delta$, where $m_{\mathcal{H}}(2N)$ is the growth function of of the hypothesis set $\mathcal{H}$.
- Another valida bound on the out-of-sample error is

$$E_{\text{out}}(g) \leq E_{\text{in}}(g) + \sqrt{\frac{8}{N} \log \frac{4((2N)^{d_{VC}} + 1)}{\delta}}, \tag{2}$$

  where d_{VC} is the so called *Vapnik-Chervonenkis* dimension of $\mathcal{H}$.

- The growth function $m_{\mathcal{H}}$ of a hypothesis space helps quantify a hypothesis set's capacity or complexity. It provides a measure of how many distinct ways a set of points can be classified by the hypotheses in the class.
- The VC dimension provides a way to quantify the complexity of a hypothesis set. A higher VC dimension means the hypothesis class is more complex and can express a greater variety of functions. Although the bounds in (2) and (3) are loose, they provide a principled framework to characterize the feasibility of learning.

Sample Complexity

The sample complexity denotes how many training examples N are needed to achieve a specific generalization performance. The performance is specified by two parameters ϵ and δ. The error tolerance ϵ denotes the allowed generalization error, and the δ determines how often the error tolerance ϵ is violated. How fast N grows as ϵ and δ become smaller indicates how much data are needed for good generalization.

We can use the VC bound to estimate the sample complexity for a given learning model. Fix $\delta > 0$, and suppose we want the general-

ization error to be at most ϵ. We can show that

$$N \geq \frac{8}{\epsilon^2} \log \left(\frac{4 m_{\mathcal{H}}(2N)}{\delta} \right)$$

suffices to obtain generalization error ϵ (with probability at most $1 - \delta$). This gives an implicit bound fro the sample complexity N, since N appears on both sides of the inequality.

If we replace $m_{\mathcal{H}}(2N)$ by an upper bound based on the VC dimension, we get a similar bound

$$N \geq \frac{8}{\epsilon^2} \log \left(\frac{4((2N)^{d_{VC}} + 1)}{\delta} \right). \tag{3}$$

We can use iterative methods to obtain a numerical value for N.

Box 2: Example

Suppose we have a learning model with $d_{VC} = 3$ and would like the generalization error to be at most 0.1 with 90% confidence (so $\epsilon = 0.1$ and $\delta = 0.1$). How big is the data set we need?
Using an iterative method to solve (4), we obtain $N \approx 30,000$. If $d_VC = 4$, a similar calculation will find $N \approx 40,000$. For $d_VC = 4$, we get $N \approx 50,000$. You can see that que inequality suggests that the number of examples is approximately proportional to the VC dimension, as has been observed in practice. The proportionality constant it suggests is $10,000$, which is a gross overestimate; a more practical constant of proportionality is closer to 10.

Penalty for Model Complexity

sample complexity fixes parameters ϵ (generalization error) and δ (confidence parameter), and estimate how many examples N are needed. In practice, we are given a $\mathcal{D}$ data set, so N is fixed. In this case, the relevant question is what performance we can expect given this particular N. The bounds (2) and (3) give us the answer.

Box 3: Example

Suppose that $N = 100$ and we have 90% onfidence requirement ($\delta = 1$). We could ask what error bar we can offer with this confidence if $\mathcal{H}$ has $d_{VC} = 1$. Using (4) we have

$$E_{\text{out}}(g) \leq E_{\text{in}}(g) + \sqrt{\frac{8}{100} \log \frac{4(201)}{0.1}} \approx E_{\text{in}}(g) + 0.848,$$

with confidence $\geq$ 100%, which is a poor bound on $E_{\text{out}}(g)$. Even if $E_{\text{in}}(g) = 0$, $E_{\text{out}}(g)$ may still be close to 1. If $N = 1000$, the we get $E_{\text{out}}(g) \leq E_{\text{in}}(g) + 0.301$, a more suitable bound.

The bound in (2) can be written as

$$E_{\text{out}}(g) \leq E_{\text{in}}(g) + \Omega(N, \mathcal{H}, \delta).$$

where

$$\begin{aligned} \Omega(N, \mathcal{H}, \delta) &= \sqrt{\frac{8}{N} \log \frac{4 m_{\mathcal{H}}(2N)}{\delta}} \\ &\leq \sqrt{\frac{8}{N} \log \frac{4((2N)^{d_{VC}} + 1)}{\delta}}. \end{aligned}$$

The function $\Omega(N, \mathcal{H}, \delta)$ is a penalty for model complexity.

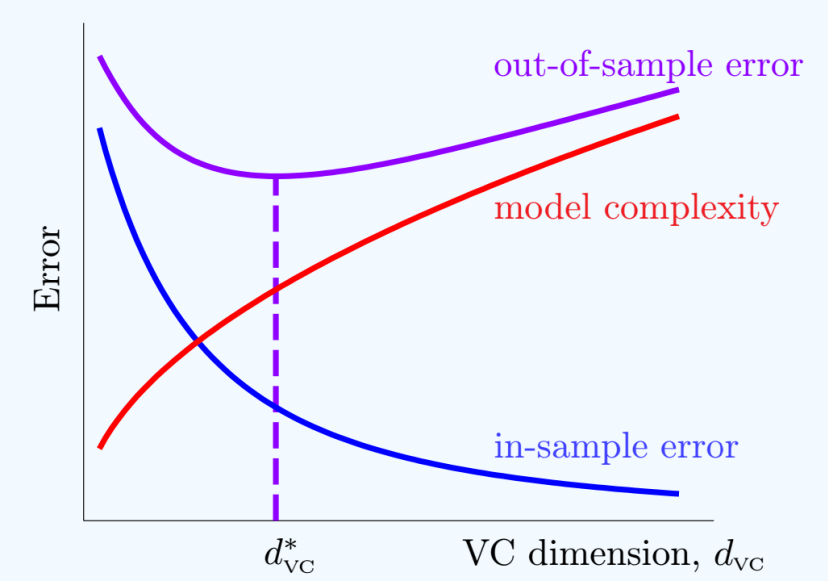


Figure 10: When we use a more complex learning model that has a higher VC dimension d_{VC}, we are likely to fit the training data better, resulting in lower in-sample error. Still, we pay a higher penalty for higher model complexity. A combination of the two estimates the out-of-sample error thus attains a minimum at some intermediate d^*_{VC}

.

The Test Set

A *tes set* is a data set not involved in the training process. The final hypothesis g is evaluated on the test set, and the result is an estimate

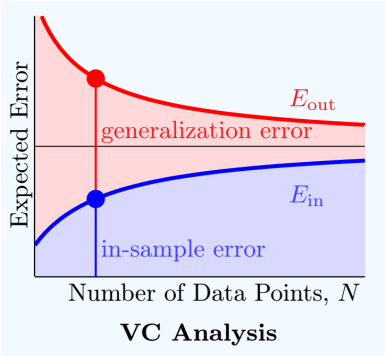


Figure 11: Learning curveE_{in}.

of $E_{\text{out}}(g)$. The error we get on the test set is called E_{test}. The Hoeffding Inequality can be used to bound $E_{\text{out}}(g)$. This is a much tighter bound than the VC bound.

$$P(|\ E_{\text{out}}(g) - E_{\text{test}}(g)\ | \geq \epsilon) \leq 2\exp^{-2\epsilon^2 N}$$

For example, if you have $1,000$ data points in the test set, E_{test} will be within 5% of $E_{\text{out}}(g)$ with probability $\geq 98\%$. The bigger the test set you use, the more accurate $E_{\text{test}}(g)$ will be of $E_{\text{out}}(g)$.

Learning Curve

The *learning curve* summarizes the behavior of the in-sample and out-sample errors as we vary the size of the training set.

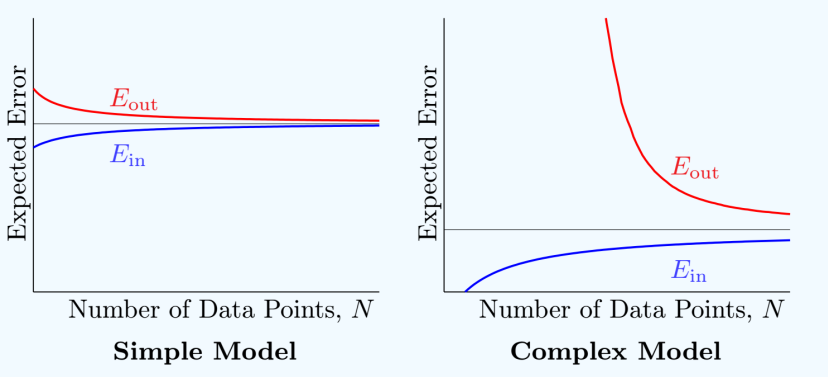


Figure 12: learning curveE_{in}.

Box 4: Training, Testing and Validation Errors

- The *out-of-sample error (testing error)* E_{out} measures how well our training data has *generalized* to data that we have not seen before. E_{out} is based on the performance over the entire feature (input space). Intuitively, if we want to estimate the value of E_{out} using a sample of data points, these points must be new test points that have not been used for training.
- The *in-sample error (training error)* E_{in}, by contrast, is based on data points that have been used for training. It expressly measures training performance.
- The *generalization error* is defined here as the discrepancy between E_{in} and E_{out}. A small generalization error suggests that the learning algorithm can generalize over new data.
- The *validation set* is similar to the test set. However, it is not directly used for training. It is used to make certain decisions during the learning process. The *validation error* is denoted as E_{val}.
- The validation set is created as follows. The data set of size N is partitioned into the training set of size $(N - K)$ and a validation set of size K.
- Notice that the minute a set affects the learning process in any way is no longer a test set. However, we will show that the way the validation set is used in the learning process is so benign that the estimation of E_{out} remains almost intact.

Appendix: Information Theory

The following are important concepts in Information Theory.

Definition 2: Entropy

The *entropy* of a discrete random variable X is defined by

$$\mathrm{H}(p_X) = -\sum_{\alpha} p_X(\alpha) \log_2 p_X(\alpha) = \mathrm{E}\left[\log_2 \frac{1}{p_X(x)}\right],$$

where $H(X) \geq 0$. Here, we use the convention $0 \log 0 = 0$.

Definition 3: Kullback Leibler Distance

The relative entropy or Kullback Leibler distance between two probability mass functions p_X and p_Y

$$\mathrm{D}_{KL}(p_X \,||\, p_Y) = \sum_{\omega} p_X(\omega) \log_2 \frac{p_X(\omega)}{p_Y(\omega)} = \mathrm{E}_X\left[\log_2 \frac{p_X(x)}{p_Y(y)}\right].$$

Here, we use the convention

$$0 \log_2 \frac{0}{p_Y(x)} = 1,$$

and $p_X(x) \log_2 \frac{p_X(x)}{0} = \infty$.

Definition 4: Cross Entropy (or Relative Entropy)

The cross-entropy of the distribution p_X relative to the distribution p_Y is defined as

$$\mathrm{H}(p_X, p_Y) = -\sum_{\omega} p_X(\omega) \log_2 p_Y(\omega) = -\mathrm{E}_X[\log_2 p_Y(y)],$$

which can be written as

$$\mathrm{H}(p_X, p_Y) = \mathrm{H}(p_X) + \mathrm{D}_{KL}(p_X \,||\, p_Y).$$

Definition 5: Asymptotic Equipartition Property

In information theory, the analog of the law of large numbers is the Asymptotic Equipartition Property. This property is formalized as follows.
If $X_1, X_2, \ldots, X_N$ ar *i.i.d.* $\sim p_X$, then

$$\lim_{N\to\infty} -\frac{1}{N} p(x_1, x_2, \ldots, x_N) = \lim_{N\to\infty} -\frac{1}{N} \prod_{n=1}^{N} p(X_n) \to H(X)$$

in probability.